Differential Privacy in Data Analytics

Course Code and Term	CS4413 Winter 2026
Selected Project Name/Topic	Differential Privacy in Data Analytics
Date of Submission	21/02/2026
Team Members and Student IDs	- Khaled Al Tamimi 3766938 - Omar Mattar 3750184 - Afif Saba 3682801

Abstract

In order to achieve differential privacy when releasing statistical information from datasets, this project assesses and contrasts the Laplace and Gaussian mechanisms. Measuring the accuracy-privacy tradeoff under various privacy budgets (ε values) is the aim. We apply both mechanisms to release counts, means, and histograms using an actual dataset. We use mean squared error and absolute error to assess utility. Both mechanisms have undergone initial implementation, and initial experiments with varying ε values have been carried out. Future research will examine privacy utility tradeoffs in greater detail and visualize them.

Goals and Scope

The goal of this project is to experimentally compare:

- Laplace Mechanism (ε-DP)
- Gaussian Mechanism (ε, δ-DP)

in the context of releasing statistical summaries from a dataset.

We focus on evaluating:

- Accuracy degradation as ε decreases
- Differences in noise behavior
- Which mechanism performs better for different query types

Use Case

We simulate a scenario where a university wants to release anonymized statistics about students without revealing individual data.

We apply DP mechanisms to:

1. Release total count
2. Release average GPA (mean query)
3. Release histogram of grades

Related Work

Differential privacy provides a guarantee that changing a single individual's data included in the analysis will not affect the result much. The seminal work of Dwork et al. in 2006 established the concept of ε-differential privacy. They showed that differential privacy can be achieved by adding noise to the query result, which is proportional to the global sensitivity of the query. This concept of differential privacy provides a direct motivation for the current evaluation of how accuracy decreases as ε decreases in the context of simple statistics such as count or histogram, where global sensitivity can be easily estimated.

For numerical queries, ε-differential privacy may not be optimal. In such a scenario, the Gaussian mechanism is usually adopted because it provides a guarantee of (ε, δ)-DP. A method of analytically choosing the scale of the Gaussian noise in the context of L2 sensitivity was provided by Balle and Wang in 2018. This method is important in adopting the Gaussian mechanism in a principled way rather than relying on ad hoc tuning. When global sensitivity is high, it may be too conservative. Techniques such as smooth sensitivity and sample-and-aggregate were provided by Nissim, Raskhodnikova, and Smith in 2007.

In view of the fact that DP guarantees degrade with the use of multiple outputs, privacy accounting becomes a crucial aspect. Mironov (2017) proposed Rényi Differential Privacy (RDP), which offers a

better description of the overall privacy loss, especially when composing the Gaussian mechanism, compared to the naïve ε, δ-DP composition. Additionally, NIST SP 800-226 (2025) offers useful recommendations for the evaluation and reporting of DP guarantees, such as the use of privacy units, the interpretation of ε and δ, and common mistakes to avoid, in a standard and defensible manner.

Lastly, the implementation of DP mechanisms becomes a crucial aspect. Google's Differential Privacy libraries provide useful building blocks and examples for the deployment of DP mechanisms, whereas Haney et al. (2022) highlight the potential pitfalls of DP in practice, such as those that arise in floating-point arithmetic, which can impact the theoretical guarantees, thus motivating the experimental work.

Methodology

Dataset*: We use the Adult Income dataset obtained from the UCI Machine Learning Repository, which contains demographic and employment-related attributes derived from the 1994 U.S. Census. After removing records with missing values, the dataset consists of approximately 30,162 records across 14 features. The attributes used in this study are age, hours-per-week, and income classification (above or below $50K annually).*
Queries/tasks*: Three aggregate queries are evaluated: a count query (number of individuals earning above $50K), a mean query (average age), and a histogram of education levels. These query types were selected as they represent common real-world analytics tasks with well-defined sensitivity properties.*
Sensitivity bounding plan*: For the count query, global sensitivity is $\Delta f = 1$, as adding or removing one individual changes the count by at most one. For the mean age query, sensitivity is computed as (max age − min age) / n, where n is the number of records, bounding the influence of any single individual on the result.*
Mechanisms*: The Laplace mechanism is applied to achieve ε-differential privacy by adding noise drawn from a Laplace distribution scaled to $\Delta f/\varepsilon$. The Gaussian mechanism will be applied to achieve (ε, δ)-differential privacy, with δ fixed at 10^{-5}, adding noise drawn from a normal distribution scaled accordingly.*
Evaluation*: Both mechanisms are evaluated across $\varepsilon \in \{0.1, 0.5, 1.0, 2.0\}$. Utility is measured using absolute error between the true and privatized query results. Results are compared across mechanisms and query types to assess the privacy-utility tradeoff.*
Tools*: All experiments are implemented in Python 3 using NumPy for noise generation and pandas for data loading and preprocessing. Development is conducted locally in Visual Studio Code.*

Work Completed Table

Task	Owner	Status	Evidence
Risk and Timeline Table	Omar	Done	In report

Researching documents	Afif	In Progress	In report
Running experiments	Khaled	In Progress	In report

Source code:

```
import pandas as py
import numpy as np


# Laplace mechanism
def laplace_mechanism(trueCount, sensitivity, epsilon):
    noise = np.random.laplace(0, sensitivity / epsilon)
    return trueCount + noise

#Gaussian mechanism to be implemented

def main():

    cols = ["age", "workclass", "fnlwgt", "education",
"education-num","marital-status", "occupation", "relationship", "race",
"sex",
    "capital-gain", "capital-loss", "hours-per-week", "native-country",
"income"]

    df =
py.read_csv(r"C:\Users\Khaled\Downloads\CS4413\Project\Main\adult.data",
names=cols, na_values="?", skipinitialspace=True)
    df = df.dropna()

    #Showed me the exact strings stored in the income column, so that i
could match them precisely in the comparison.
    print(df["income"].unique(), "\n")

    #Ordinary Queries
    over50kSalaries = (df["income"]==">50K").sum()
    under50kSalaries = (df["income"]=="<=50K").sum()
    meanAge = df["age"].mean()
    meanHours = df["hours-per-week"].mean()
```

```
    print("Count >50k:", over50kSalaries)
    print("Count <=50k:", under50kSalaries)
    print(f"Mean age: {meanAge:.2f}")
    print(f"Mean hours: {meanHours:.2f}")



    # Count query: sensitivity = 1
    epsilon_values = [0.1, 0.5, 1.0, 2.0]

    print("\nLaplace mechanism applied on Over 50K salaries count with
ranging epsilon from 0.1 to 2 and a sensitivity of 1\n")

    print("epsilon | DP Count | Absolute Error")
    print("-" * 40)
    for eps in epsilon_values:
        dp_count = laplace_mechanism(over50kSalaries, sensitivity=1,
epsilon=eps)
        error = abs(over50kSalaries - dp_count)
        print(f"  {eps}   |  {dp_count:.1f}  |  {error:.1f}")

    print("\nLaplace mechanism applied on mean age with ranging epsilon
from 0.1 to 2 \n")
    minAge =  (df["age"].min())
    maxAge = (df["age"].max())
    ageSensitivity = (maxAge - minAge)/len(df)
    print("epsilon | DP Count | Absolute Error")
    print("-" * 40)
    for eps in epsilon_values:
        dp_count = laplace_mechanism(meanAge, sensitivity=ageSensitivity,
epsilon=eps)
        error = abs(meanAge - dp_count)
        print(f"  {eps}   |  {dp_count:.1f}  |  {error:.1f}")



main()
```

Terminal Output:

```
PS C:\Users\Khaled\AppData\Local\Programs\Microsoft VS Code> & C:\Python314\python.exe c:/Users/Khaled/Downloads/CS4413/Project/Main/dataProcessing.py
<StringArray>
['<=50K', '>50K']
Length: 2, dtype: str

Count >50k: 7508
Count <=50k: 22654
Mean age: 38.44
Mean hours: 40.93

Laplace mechanism applied on Over 50K salaries count with ranging epsilon from 0.1 to 2 and a sensitivity of 1

epsilon | DP Count | Absolute Error
---------------------------------------
  0.1   |  7517.3  |  9.3
  0.5   |  7509.0  |  1.0
  1.0   |  7507.9  |  0.1
  2.0   |  7507.8  |  0.2

Laplace mechanism applied on mean age with ranging epsilon from 0.1 to 2

epsilon | DP Count | Absolute Error
---------------------------------------
  0.1   |  38.4  |  0.0
  0.5   |  38.4  |  0.0
  1.0   |  38.4  |  0.0
  2.0   |  38.4  |  0.0
PS C:\Users\Khaled\AppData\Local\Programs\Microsoft VS Code>
```

Risk and Timeline Table

Risk Table:

Risk	Impact	Likelihood	Mitigation Strategy
Incorrect sensitivity calculation	Invalid privacy guarantees	Medium	Carefully derive sensitivity for count ($\Delta f=1$), mean (bounded domain), histogram; verify with literature
Improper Gaussian parameter (σ) calibration	Weak (ε,δ)-DP guarantee	Medium	Use standard Gaussian mechanism formula and clearly fix δ (e.g., 10^{-5})
Too much noise at small ε leading to unusable results	Low utility	High	Test multiple ε values (0.1–2) and report tradeoff clearly
Implementation bugs in noise addition	Wrong results	Medium	Validate outputs on small synthetic dataset with known values
Time constraint before final submission	Incomplete evaluation	Low	Finish implementation early; dedicate final phase to analysis/plots
Dataset issues (unbounded values affecting sensitivity)	Incorrect noise scale	Medium	Clip/bound data before applying DP

Timeline table:

Week	Task
Week 1	Finalize dataset and define queries (count, mean, histogram)
Week 2	Implement Laplace mechanism
Week 3	Implement Gaussian mechanism (ε, δ)
Week 4	Run experiments for ε = 0.1, 0.5, 1, 2
Week 5	Compute error metrics (Absolute Error, MSE)
Week 6	Generate privacy–utility graphs
Week 7	Analyze results and compare mechanisms
Week 8	Write the final report and polish the presentation.

The timeline table is a rough guideline subject to change based on experimental challenges or time constraints.

References

[1] C. Dwork, F. McSherry, K. Nissim, and A. Smith, "Calibrating noise to sensitivity in private data analysis," in *Theory of Cryptography (TCC 2006)*, Lecture Notes in Computer Science, vol. 3876, pp. 265–284, 2006, doi: 10.1007/11681878_14.

[2] K. Nissim, S. Raskhodnikova, and A. Smith, "Smooth sensitivity and sampling in private data analysis," in *Proc. ACM Symp. Theory of Computing (STOC)*, pp. 75–84, 2007, doi: 10.1145/1250790.1250803.

[3] B. Balle and Y.-X. Wang, "Improving the Gaussian mechanism for differential privacy: Analytical calibration and optimal denoising," in *Proc. 35th Int. Conf. Mach. Learn. (ICML)*, Proc. Mach. Learn. Res., vol. 80, pp. 394–403, 2018.

[4] I. Mironov, "Rényi differential privacy," in *Proc. IEEE 30th Computer Security Foundations Symp. (CSF)*, pp. 263–275, 2017, doi: 10.1109/CSF.2017.11.

[5] J. Near, D. Darais, N. Lefkovitz, and G. Howarth, *Guidelines for Evaluating Differential Privacy Guarantees*, NIST Special Publication 800-226, Mar. 2025, doi: 10.6028/NIST.SP.800-226.

[6] OpenDP, "A framework to understand DP," *OpenDP Documentation*. Accessed: Feb. 19, 2026. [Online].

[7] Google, “Google’s differential privacy libraries,” *GitHub repository (google/differential-privacy)*. Accessed: Feb. 19, 2026. [Online].

[8] S. Haney, D. Desfontaines, L. Hartman, R. Shrestha, and M. Hay, “Precision-based attacks and interval refining: how to break, then fix, differential privacy on finite computers,” *arXiv:2207.13793*, 2022. [Online].